Processingをはじめよう

第12章

データ

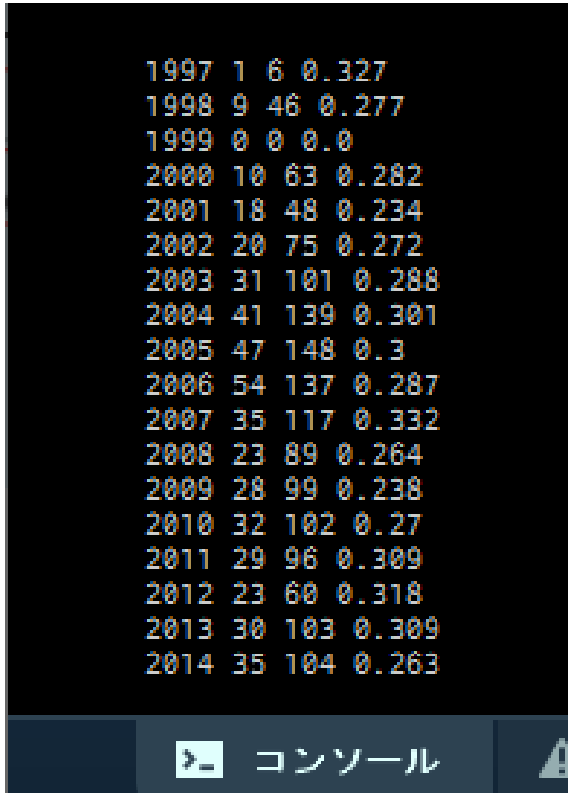
目次

- データ構造
- Tableクラスと表
- JSON
- インターネット上のデータとAPI
- Robot 10: Data

Tableクラスと表

- Example 12-1 表を読む

```
Table stats;

void setup() {
  stats = loadTable("ortiz.csv");

  for(int i=0 ; i<stats.getRowCount() ; i++){
    int year = stats.getInt(i, 0);
    int homeRuns = stats.getInt(i, 1);
    int rbi = stats.getInt(i, 2);
    float average = stats.getFloat(i, 3);
    println(year, homeRuns, rbi, average);
  }
}
```



- media.zipを展開してortiz.csvを追加

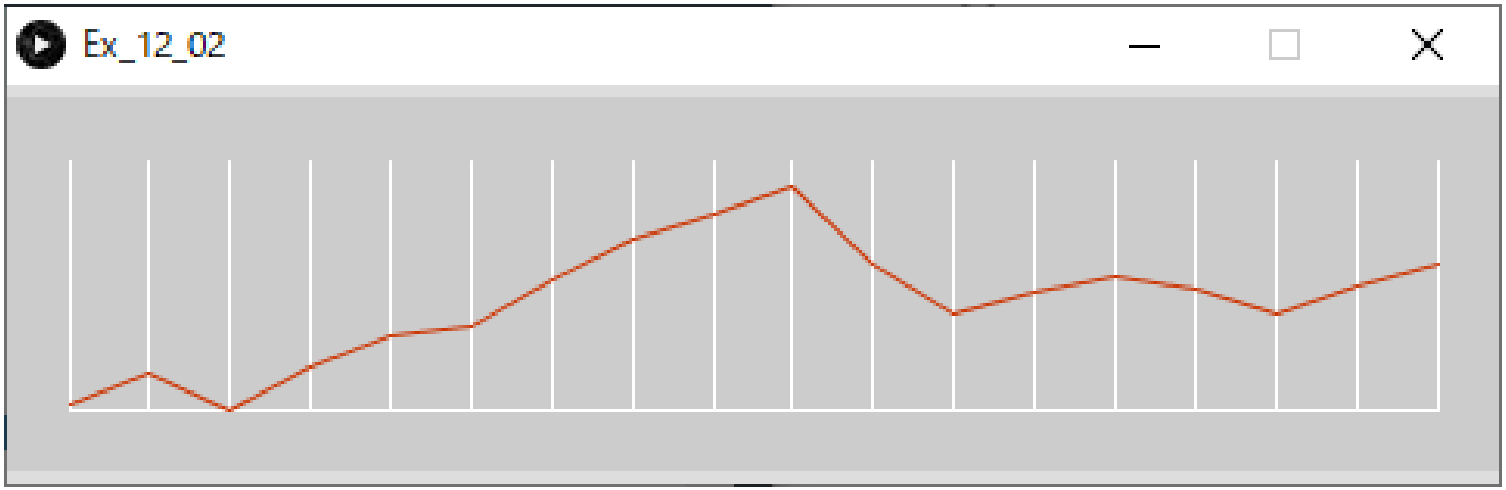
Tableクラスと表

- Example 12-2 表を読む

```
int[] homeRuns;

void setup(){
  size(480, 120);
  Table stats = loadTable("Ortiz.csv");
  int rowCount = stats.getRowCount();
  homeRuns = new int[rowCount];
  for(int i=0 ; i<homeRuns.length ; i++){
    homeRuns[i] = stats.getInt(i, 1);
  }
}

void draw(){
  background(204);
  stroke(255);
  line(20, 100, 20, 20);
  line(20, 100, 460, 100);
  for(int i=0 ; i<homeRuns.length ; i++){
    float x = map(i, 0, homeRuns.length, 20, 460);
    line(x, 20, x, 100);
  }
```

Tableクラスと表

```
  noFill();
  stroke(204, 51, 0);

  beginShape();
  for(int i=0 ; i<homeRuns.length ; i++){
    float x = map(i, 0, homeRuns.length-1, 20, 460);
    float y = map(homeRuns[i], 0, 60, 100, 20);
    vertex(x, y);
  }
  endShape();
}
```

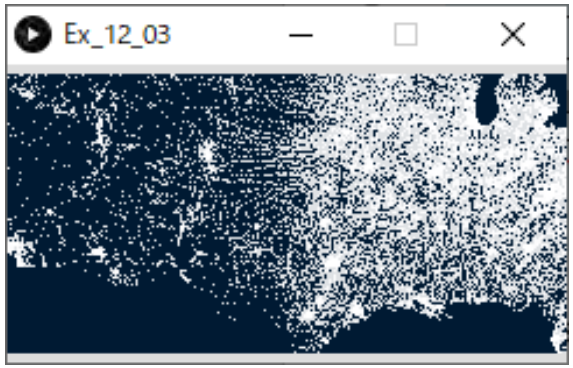


Tableクラスと表

- Example 12-3 29,740の町

```
Table cities;

void setup() {
  size(240, 120);
  cities
    = loadTable("cities.csv",
                "header");
  stroke(255);
}
```

```
void draw() {
  background(0, 26, 51);
  float xoffset = map(mouseX, 0,
          width, -width*3, -width);
  translate(xoffset, -300);
  scale(10);
  strokeWeight(0.1);
  for (int i = 0; i < cities.getRowCount(); i++) {
    float latitude = cities.getFloat(i, "lat");
    float longitude = cities.getFloat(i, "lng");
    setXY(latitude, longitude);
  }
}

void setXY(float lat, float lng) {
  float x = map(lng, -180, 180, 0, width);
  float y = map(lat, 90, -90, 0, height);
  point(x, y);
}
```



Tableクラス

- https://processing.org/reference/Table.html

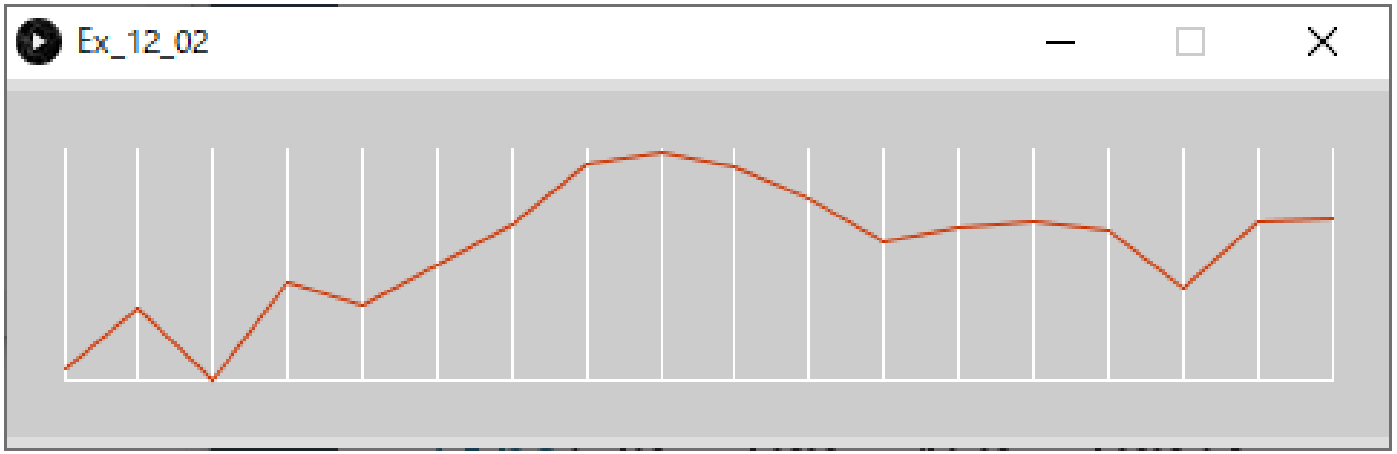
メソッド名	内容
addColumn()	新しい列を追加
removeColumn()	列を削除
getColumnCount()	列数を取得
getRowCount()	行数を取得
clearRows()	すべての行を削除
addRow()	行を追加
removeRow()	行を削除
getRow()	行を取得
rows()	複数の行を取得
getInt()	指定した行と列から整数値を取得
setInt()	指定した行と列に整数値を格納
getFloat()	指定した行と列からfloat値を取得
setFloat()	指定した行と列にfloat値を格納

メソッド名	内容
getString()	指定した行と列から文字列を取得
setString()	指定した行と列に文字列を格納
getStringColumn()	指定した列のすべての値を取得
findRow()	指定した値を含む行を検索
findRows()	指定した値を含むすべての行を検索
matchRow()	指定した式に一致する行を検索
matchRows()	指定した式に一致する複数の行を検索
removeTokens()	文字を削除
trim()	値から空白を削除
sort()	並べ替える

Kadai_12_01

- Example 12-1、2を参考に、rbiの値を折れ線グラフで表示する
  - 縦軸の上限を150に変更

```
float y = map(rbi[i], 0, 150, 100, 20);
```



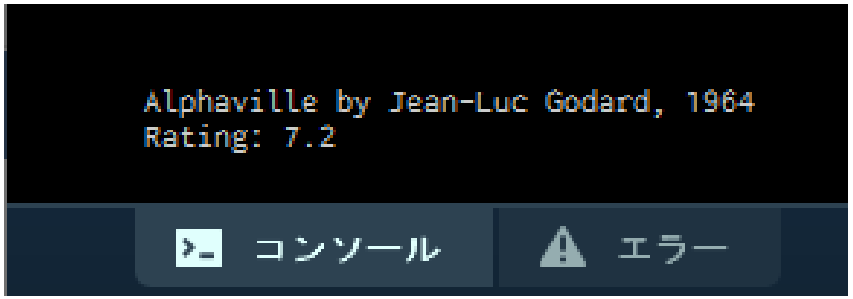
JSON

- Example 12-4 JSONファイルを読む

```
JSONObject film;

void setup() {
  film = loadJSONObject("film.json");
  String title = film.getString("title");
  String dir = film.getString("director");
  int year = film.getInt("year");
  float rating = film.getFloat("rating");
  println(title + " by " + dir + ", " + year);
  println("Rating: " + rating);
}
```

- film.json

```
{
 "title": "Alphaville",
 "director": "Jean-Luc Godard",
 "year": 1964,
 "rating": 7.2
}
```



JSON（JavaScript Object Notation）

- {}の中に、”キー”と値を：で区切る
  { ”キー”：値 }
  - 複数のキーと値を，で区切る
    { ”キー1”：値1，”キー2”：値2 }
  - データ型
    - 文字列、数値、ヌル、真偽値、オブジェクト、配列

JSON

- Example 12-5 JSONファイルのデータをビジュアライズ

```
Film[] films;

void setup() {
  size(480, 120);
  JSONArray filmArray = loadJSONArray("films.json");
  films = new Film[filmArray.size()];
  for (int i = 0; i < films.length; i++) {
    JSONObject o = filmArray.getJSONObject(i);
    films[i] = new Film(o);
  }
}

void draw() {
  background(0);
  for (int i = 0; i < films.length; i++) {
    int x = i*32 + 32;
    films[i].display(x, 105);
  }
}
```

```
class Film {

  String title;
  String director;
  int year;
  float rating;

  Film(JSONObject f) {
    title = f.getString("title");
    director = f.getString("director");
    year = f.getInt("year");
    rating = f.getFloat("rating");
  }

  void display(int x, int y) {
    float ratingGray = map(rating,
                   6.5, 8.1, 102, 255);
    pushMatrix();
    translate(x, y);
    rotate(-QUARTER_PI);
    fill(ratingGray);
    text(title, 0, 0);
    popMatrix();
  }
}
```

インターネット上のデータとAPI

- Example 12-6 気温のデータを抽出する

```
void setup() {
  float temp = getTemp("cincinnati.json");
  println(temp);
}

float getTemp(String fileName) {
  JSONObject weather = loadJSONObject(fileName);
  JSONArray list = weather.getJSONArray("list");
  JSONObject item = list.getJSONObject(0);
  JSONObject main = item.getJSONObject("main");
  float temperature = main.getFloat("temp");
  return temperature;
}
```

```
{
 "message":"accurate", "cod":"200", "count":1,
 "list":
  [
   {"id":4508722,"name":"Cincinnati","coord":{"lon":-84.456886,"lat":39.161999},
    "main":{ "temp":34.16, "temp_min":34.16,"temp_max":34.16,"pressure":999.98,"sea_level":1028.34,"grnd_level":999.98,"humidity":77},
    "dt":1423501526, "wind":{"speed":9.48,"deg":354.002}, "sys":{"country":"US"}, "clouds":{"all":80},
    "weather":[{"id":803,"main":"Clouds","description":"broken clouds","icon":"04d"}]
   }
  ]
}
```

インターネット上のデータとAPI

- Example 12-7 メソッドの連結

```
JSONObject weather;

void setup() {
  float temp = getTemp("cincinnati.json");
  println(temp);
}

float getTemp(String fileName) {
  JSONObject weather = loadJSONObject(fileName);
  return weather.getJSONArray("list").getJSONObject(0).getJSONObject("main").getFloat("temp");
}
```

```
{
 "message":"accurate", "cod":"200", "count":1,
 "list":
  [
   {"id":4508722,"name":"Cincinnati","coord":{"lon":-84.456886,"lat":39.161999},
    "main":{ "temp":34.16, "temp_min":34.16,"temp_max":34.16,"pressure":999.98,"sea_level":1028.34,"grnd_level":999.98,"humidity":77},
    "dt":1423501526, "wind":{"speed":9.48,"deg":354.002}, "sys":{"country":"US"}, "clouds":{"all":80},
    "weather":[{"id":803,"main":"Clouds","description":"broken clouds","icon":"04d"}]
   }
  ]
}
```

JSONObject

- https://processing.org/reference/JSONObject.html

メソッド名	内容
getString()	文字列を取得する
getInt()	int値を取得する
getFloat()	float値を取得する
getBoolean()	boolean値を取得する
getJSONArray()	JSON配列を取得する
getJSONObject()	JSONオブジェクトを取得する
isNull()	値がヌルかどうか判別する
setString()	文字列を格納する
setInt()	int値を格納する
setFloat()	float値を格納する
setBoolean()	boolean値を格納する
setJSONArray()	JSON配列を設定する
setJSONObject()	JSONオブジェクトを設定する

JSONArray

- https://processing.org/reference/JSONArray.html

メソッド名	内容
getString()	文字列を取得する
getInt()	int値を取得する
getFloat()	float値を取得する
getBoolean()	boolean値を取得する
getJSONArray()	JSON配列を取得する
getJSONObject()	JSONオブジェクトを取得する
getStringArray()	配列全体を文字列の配列として取得する
getIntArray()	配列全体をint配列として取得する
append()	値を追加して配列の長さを１増やす
setString()	文字列を格納する
setInt()	int値を格納する
setFloat()	float値を格納する

メソッド名	内容
setBoolean()	boolean値を格納する
setJSONArray()	JSON配列を設定する
setJSONObject()	JSONオブジェクトを設定する
size()	要素数を取得する
isNull()	値がヌルかどうか判別する
remove()	要素を削除する

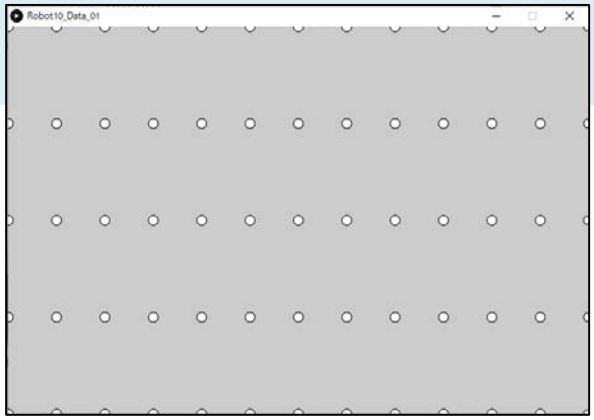
Kadai_12_02

- Example 12-6, 7を参考に、風速のデータを抽出する


```
{
 "message":"accurate", "cod":"200", "count":1,
 "list":
  [
   {
    "id":4508722,
    "name":"Cincinnati",
    "coord":{"lon":-84.456886,"lat":39.161999},
    "main":
    {
     "temp":34.16,
     "temp_min":34.16,
     "temp_max":34.16,
     "pressure":999.98,
     "sea_level":1028.34,
     "grnd_level":999.98,
     "humidity":77
    },
    "dt":1423501526,
    "wind":
    {
     "speed":9.48,
     "deg":354.002
    },
    "sys":{"country":"US"},
    "clouds":{"all":80},
    "weather":
     [
      {
       "id":803,
       "main":"Clouds",
       "description":"broken clouds",
       "icon":"04d"
      }
     ]
   }
  ]
}
```

Robot 10: Data

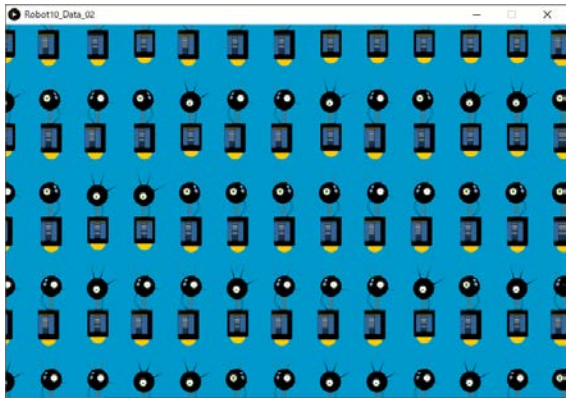
```
// Robot10_01
PrintWriter output;

void setup() {
  size(720, 480);
  output = createWriter("botArmy.tsv");
  output.println("type¥tx¥ty");
  for (int y = 0; y <= height; y += 120) {
    for (int x = 0; x <= width; x += 60) {
      int robotType = int(random(1, 4));
      output.println(robotType + "¥t" + x + "¥t" + y);
      ellipse(x, y, 12, 12);
    }
  }
  output.flush();
  output.close();
}
```

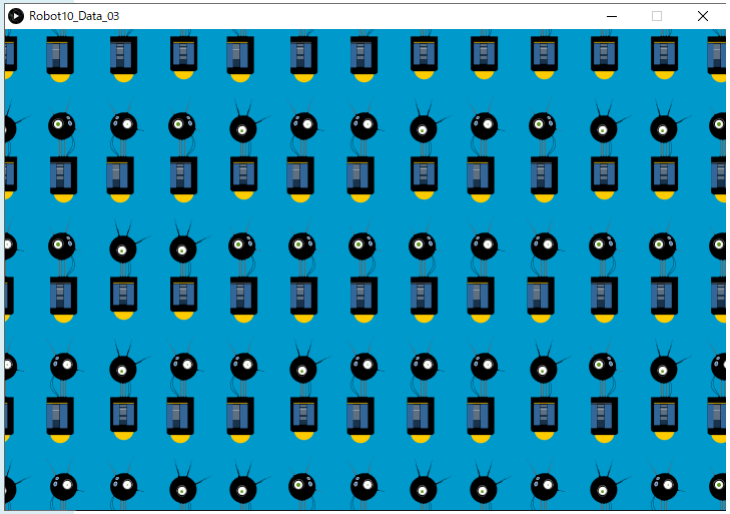


Robot 10: Data

```
// Robot10_02
Table robots;
PShape bot1;
PShape bot2;
PShape bot3;

void setup() {
  size(720, 480);
  background(0, 153, 204);
  bot1 = loadShape("robot1.svg");
  bot2 = loadShape("robot2.svg");
  bot3 = loadShape("robot3.svg");
  shapeMode(CENTER);
```

```
  robots = loadTable("botArmy.tsv", "header");
  for (int i = 0; i < robots.getRowCount(); i++) {
    int bot = robots.getInt(i, "type");
    int x = robots.getInt(i, "x");
    int y = robots.getInt(i, "y");
    float sc = 0.3;
    if (bot == 1) {
      shape(bot1, x, y, bot1.width*sc, bot1.height*sc);
    } else if (bot == 2) {
      shape(bot2, x, y, bot2.width*sc, bot2.height*sc);
    } else {
      shape(bot3, x, y, bot3.width*sc, bot3.height*sc);
    }
  }
}
```



Robot 10: Data

```
// Robot10_03
int numRobotTypes = 3;
PShape[] shapes = new PShape[numRobotTypes];
float scalar = 0.3;

void setup() {
  size(720, 480);
  background(0, 153, 204);
  for (int i = 0; i < numRobotTypes; i++) {
    shapes[i] = loadShape("robot" + (i+1) + ".svg");
  }
  shapeMode(CENTER);
  Table botArmy = loadTable("botArmy.tsv", "header");
  for (TableRow row : botArmy.rows()) {
    int robotType = row.getInt("type");
    int x = row.getInt("x");
    int y = row.getInt("y");
    PShape bot = shapes[robotType - 1];
    shape(bot, x, y, bot.width*scalar, bot.height*scalar);
  }
}
```